

WiseTrip — Documentación técnica

Curso: Fundamentos de Ingeniería de Software

Grupo: FIS_2630_1204_G3

Sprint: 3

Historias implementadas: HU#22, HU#23, HU#24, HU#25, HU#26, HU#27

1. Qué se hizo

El punto de partida fue un prototipo de WiseTrip escrito en React (`Boceto_wisetrip.jsx`), que funcionaba como una maqueta visual: todas las pantallas existían, pero los datos vivían únicamente en el estado del navegador y no había lógica de servidor.

El trabajo de este sprint consistió en migrar ese prototipo a una **aplicación web Java real**, con arquitectura por capas, validación del lado del servidor, manejo de sesión y la integración del algoritmo de recomendación desarrollado por el equipo.

El resultado es un flujo completo y navegable: una persona se registra, inicia sesión, indica desde dónde viaja, responde un cuestionario de preferencias, define fechas y presupuesto, y recibe tres destinos recomendados ordenados por qué tanto coinciden con su perfil.

2. Tecnologías utilizadas

Componente	Versión	Rol en el proyecto
Java	21	Lenguaje base
Spring Boot	4.1.1	Framework de aplicación
Spring MVC	(Spring Framework 7)	Patrón Modelo-Vista-Controlador
JSP (Jakarta Pages)	4.0	Motor de vistas
JSTL	3.0	Etiquetas lógicas dentro de las vistas
Maven	Wrapper incluido	Gestión de dependencias y compilación
Apache Tomcat	11.0.24	Servidor de aplicaciones (embebido)
Jakarta EE	11	Especificación base (<code>jakarta.servlet</code>)

Empaquetado: WAR. Esta decisión no es cosmética: Spring Boot solo puede renderizar páginas JSP cuando el proyecto se empaqueta como WAR. Con empaquetado JAR, el motor Jasper no se activa y las páginas se devuelven como texto plano.

2.1 Qué es Spring Boot y por qué se usó

Spring es el framework más extendido para desarrollo empresarial en Java. Spring Boot es la capa que lo hace utilizable sin configuración manual: incluye un servidor Tomcat embebido, detecta automáticamente los componentes de la aplicación y define valores por defecto razonables para casi todo.

En la práctica, esto significa que el proyecto se ejecuta con un solo comando y no requiere instalar ni configurar un servidor aparte:

```
mvnw.cmd spring-boot:run
```

Los dos mecanismos de Spring que sostienen toda la aplicación son:

Inyección de dependencias. Las clases no crean sus propias dependencias; las reciben por constructor y Spring se encarga de proveerlas. Por ejemplo:

```
@Controller
public class AuthControlador {

    private final UsuarioServicio usuarioServicio;

    public AuthControlador(UsuarioServicio usuarioServicio) {
        this.usuarioServicio = usuarioServicio;
    }
}
```

Spring crea una única instancia de `UsuarioServicio` (por la anotación `@Service`) y la entrega a todos los controladores que la necesiten. Esa unicidad es lo que permite que un usuario registrado en un controlador sea visible desde otro.

Mapeo de rutas por anotaciones. Cada método indica qué dirección atiende y con qué verbo HTTP:

```
@GetMapping("/registro")    // muestra el formulario
@PostMapping("/registro")  // procesa el envío
```

2.2 Cómo se conectan Java y las vistas

La configuración en `application.properties` define dónde buscar las páginas:

```
spring.mvc.view.prefix=/WEB-INF/vistas/
spring.mvc.view.suffix=.jsp
```

Cuando un controlador devuelve la cadena `"login"`, Spring compone la ruta `/WEB-INF/vistas/login.jsp` y renderiza ese archivo. El controlador nunca genera HTML: solo decide qué vista mostrar y qué datos entregarle.

Los datos viajan del controlador a la vista mediante el objeto `Model`:

```
model.addAttribute("mensaje", "Planea tu viaje...");
```

Y se leen en el JSP con la sintaxis de Expression Language:

```
<p>${mensaje}</p>
```

JSTL aporta la lógica de presentación. `<c:if>` reemplaza los condicionales que en React se escribían con `&&`, y `<c:forEach>` reemplaza el `.map()`:

```
<c:forEach var="categoria" items="${categorias}">
    <h2>${categoria.nombre}</h2>
</c:forEach>
```

Esto tiene una consecuencia de diseño importante: las 26 preguntas del cuestionario **no están escritas en el HTML**. La vista recorre una lista que le entrega el servidor, de modo que agregar o quitar preguntas no requiere tocar la vista.

3. Arquitectura

El proyecto sigue el patrón Modelo-Vista-Controlador con tres capas de responsabilidad separada:

```
Navegador
↓
Controlador  - recibe la petición, decide qué hacer, elige la vista
↓
Servicio     - lógica de negocio, validaciones, almacenamiento
↓
Modelo       - estructuras de datos
```

3.1 Estructura de paquetes

```
com.wisetrip
├── WisetripApplication.java      Clase de arranque
│
├── controlador/
│   ├── InicioControlador       Portada
│   ├── AuthControlador         HU#22, HU#23
│   ├── ViajeControlador        HU#24
│   ├── PlanificacionControlador HU#25, HU#26, HU#27, resumen
│   └── RecomendacionControlador Algoritmo de recomendación
│
├── modelo/
│   ├── Usuario                Datos de la cuenta
│   ├── Ubicacion              Origen del viaje
│   ├── Pregunta               Clave + texto de una pregunta
│   ├── CategoriaPreferencia    Nombre + lista de preguntas
│   ├── Preferencias            Mapa de respuestas del cuestionario
│   ├── FechasViaje             Inicio y fin del viaje
│   ├── Presupuesto             Monto y moneda
│   ├── Ciudad                 Destino con sus atributos
│   ├── PreferenciasUsuario     Entrada del algoritmo
│   ├── ResultadoRecomendacion   Ciudad + puntajes
│   └── SeleccionDestinos        Los mejores destinos + mensaje
│
└── servicio/
    ├── UsuarioServicio         Registro, autenticación, validaciones
    ├── ViajeServicio           Países, ciudades, validación de origen
    ├── PreferenciasServicio     Banco de preguntas y validación
    ├── FechasServicio           Validación de fechas
    ├── PresupuestoServicio      Monedas y conversión a USD
    ├── CatalogoCiudades         Catálogo y traducción de atributos
    ├── RecomendadorDestinos     Cálculo de puntajes
    └── SelectorDestinos         Selección de los tres mejores
```

Las vistas viven en `src/main/webapp/WEB-INF/vistas/` y los estilos en `src/main/resources/static/css/`.

3.2 Almacenamiento

En esta versión los datos se guardan **en memoria**, mediante un `ArrayList` dentro de `UsuarioServicio`. Como Spring mantiene una sola instancia de cada servicio, esa lista es compartida por toda la aplicación.

Es el equivalente directo del `useState([])` que tenía el prototipo React, y tiene la misma consecuencia: los datos se pierden al detener la aplicación. La persistencia en base de datos está prevista para el siguiente sprint, y la arquitectura ya está preparada para el cambio porque toda la lógica de datos está encapsulada en los servicios.

3.3 Manejo de sesión

El progreso del usuario a través del flujo se guarda en `HttpSession`, la memoria que el servidor mantiene por cada navegador conectado:

Clave en sesión	Contenido
<code>usuarioActivo</code>	Usuario que inició sesión
<code>ubicacionOrigen</code>	Ciudad y país de partida
<code>preferenciasViaje</code>	Respuestas del cuestionario
<code>atributosSeleccionados</code>	Mapa de atributos para el algoritmo
<code>fechasViaje</code>	Fechas de inicio y regreso
<code>presupuestoViaje</code>	Monto y moneda originales
<code>presupuestoEnUsd</code>	Equivalente en dólares

Esto reemplaza al estado global que el prototipo React mantenía en su componente raíz.

4. Flujo de la aplicación

/	Portada	
/registro	→ /registro-exitoso	HU#22
/login	→	HU#23
/origen	→	HU#24
/preferencias	→	HU#25
/fechas	→	HU#26
/presupuesto	→	HU#27
/resumen	Consolidado de toda la información	
/recomendaciones	Destinos sugeridos	

Cada paso está **protegido por un guardia de sesión**. Los métodos verifican que exista el dato del paso anterior y, si no, redirigen:

```
Usuario usuario = (Usuario) sesion.getAttribute("usuarioActivo");
if (usuario == null) return "redirect:/login";
if (sesion.getAttribute("fechasViaje") == null) return "redirect:/fechas";
```

Esto impide que alguien salte pasos escribiendo la URL directamente en el navegador, y garantiza que el algoritmo siempre reciba información completa.

5. Historias de usuario implementadas

HU#22 — Registro de usuario

Formulario con seis campos y validación en el servidor. Las reglas se traspasaron íntegramente desde la función `submit()` del componente `RegisterScreen` del prototipo:

- Nombre completo obligatorio, mínimo 3 caracteres
- Tipo de documento obligatorio (CC, CE, TI, Pasaporte)
- Número de documento entre 5 y 15 dígitos, sin duplicados
- Fecha de nacimiento obligatoria, no futura, mayor de 18 años
- Correo con formato válido y sin duplicados
- Contraseña de mínimo 6 caracteres, con confirmación

La validación devuelve un `Map<String,String>` que asocia cada campo con su mensaje de error. Un mapa vacío significa que los datos son correctos:

```
public Map<String, String> validarRegistro(Usuario u, String confirmarPassword) {
    Map<String, String> errores = new LinkedHashMap<>();
    // ...
    if (edad < 18) {
        errores.put("fechaNacimiento",
            "Debes ser mayor de 18 años para crear una cuenta en WiseTrip.");
    }
    return errores;
}
```

Cuando hay errores, el controlador devuelve la misma vista con los mensajes y los datos ya escritos, de modo que la persona no pierde lo que había llenado.

Decisión de diseño: la validación es del lado del servidor, no del navegador. Las validaciones de HTML pueden desactivarse desde las herramientas de desarrollo, así que confiar en ellas sería un error de seguridad.

HU#23 — Inicio de sesión

Verificación de credenciales contra los usuarios registrados. Si coinciden, el usuario se guarda en `HttpSession` y se redirige al siguiente paso.

El mensaje de error es deliberadamente genérico: cuando el correo no existe y cuando la contraseña es incorrecta, se muestra el mismo texto. Distinguir entre ambos casos revelaría a un atacante qué correos están registrados en el sistema.

Se implementó además `/Logout`, que invalida la sesión completa.

HU#24 — Ubicación de origen

Selección de país y ciudad, con un campo opcional para el punto de partida exacto.

Las ciudades disponibles dependen del país elegido. La lista se recarga al cambiar el país mediante un envío del formulario, sin necesidad de JavaScript adicional.

La validación incluye una **verificación cruzada**: comprueba que la ciudad realmente pertenezca al país seleccionado. Esta comprobación es necesaria porque un formulario puede manipularse desde el navegador para enviar combinaciones inválidas.

HU#25 — Preferencias de viaje

Cuestionario de **26 preguntas de sí o no**, organizadas en 8 categorías: Tipo de destino, Clima, Aventura, Gastronomía, Ritmo de viaje, Cultura, Estilo de viaje y Compañía.

Decisión de arquitectura. En lugar de declarar 26 campos individuales en el modelo, las respuestas se almacenan en un `Map<String,String>` que asocia la clave de cada pregunta con su respuesta:

```
public class Preferencias {  
    private Map<String, String> respuestas = new LinkedHashMap<>();  
}
```

Spring puebla ese mapa automáticamente a partir de los nombres de los campos del formulario:

```
<input type="radio" name="respuestas[${p.clave}]" value="si">
```

El banco de preguntas vive como una lista estática en `PreferenciasServicio.CATEGORIAS`. Esto significa que **agregar, quitar o reescribir preguntas requiere editar un solo lugar**: la vista, el contador de progreso y la validación se ajustan automáticamente, porque el total se calcula en vez de escribirse a mano:

```
public int totalPreguntas() {  
    return CATEGORIAS.stream().mapToInt(c -> c.getPreguntas().size()).sum();  
}
```

Validación. No se permite continuar si falta alguna respuesta. El sistema muestra cuántas faltan y marca visualmente cada pregunta pendiente.

Experiencia de uso. La pantalla incluye una barra de progreso que indica cuántas preguntas se han respondido, y el resaltado de error de una pregunta desaparece en cuanto se responde.

Salida para el algoritmo. El método `obtenerAtributosSeleccionados()` convierte las respuestas en un `Map<String,Boolean>` que se guarda en sesión y constituye la entrada del recomendador.

HU#26 — Fechas del viaje

Captura de fecha de inicio y fecha de regreso mediante campos de tipo fecha.

Validaciones:

- Ambas fechas son obligatorias
- La fecha de inicio no puede ser anterior al día actual
- La fecha de regreso debe ser posterior a la de inicio
- La duración máxima admitida es de 60 días

La duración en días se calcula con `ChronoUnit.DAYS` y se muestra al usuario, además de quedar disponible para el cálculo del presupuesto.

HU#27 — Presupuesto y moneda

Captura del monto disponible y la moneda, con soporte para las **19 monedas de los 20 países latinoamericanos** que maneja WiseTrip.

Manejo del selector de monedas. Si el destino ya es conocido (clave `paisDestino` en sesión), el selector muestra únicamente la moneda local de ese país más el dólar como referencia. Si el destino aún no se ha definido, se muestran las 19 monedas.

Conversión a dólares. El algoritmo necesita comparar presupuestos expresados en distintas monedas contra costos de destinos en distintos países. Para resolverlo, además del monto original se calcula un equivalente aproximado en USD usando una tabla de tasas fijas, y se guarda en sesión como `presupuestoEnUsd`.

Las tasas son aproximadas y no se consultan en tiempo real. Toda la conversión está encapsulada en un único método, de modo que sustituirla por una llamada a una API de tasas de cambio no requeriría modificar nada más:

```
public double convertirAUd(Presupuesto presupuesto) {
    double valor = presupuesto.getMontoNumerico();
    Double tasa = UNIDADES_POR_USD.get(presupuesto.getMoneda());
    return Math.round((valor / tasa) * 100.0) / 100.0;
}
```

Detalle de implementación. El monto se recibe como texto y no como número. Si se declarara `double`, Spring lanzaría una excepción antes de que el código pudiera validar, y el usuario vería una página de error genérica en lugar de un mensaje comprensible.

6. Integración del algoritmo de recomendación

El algoritmo fue desarrollado por el equipo en clases independientes. La integración consistió en conectarlo al flujo web sin modificar su lógica.

6.1 Cómo funciona el cálculo

Cada ciudad recibe un puntaje entre 0 y 1, combinación ponderada de dos factores:

```
Puntaje total = 0.4 × ajuste al presupuesto + 0.6 × coincidencia de preferencias
```

Ajuste al presupuesto. Si el costo cabe en el presupuesto, el puntaje es alto y crece mientras mejor lo aproveche. Si lo excede, decae proporcionalmente al exceso.

Coincidencia de preferencias. Proporción de las preferencias marcadas con "sí" que la ciudad cumple. Si el usuario no marcó ninguna preferencia, se devuelve 1.0 para no penalizar a ningún destino.

`SelectorDestinos` ordena los resultados y devuelve los tres mejores, junto con un mensaje adecuado según la cantidad disponible.

6.2 El problema de integración y su solución

El cuestionario produce claves con la nomenclatura de las preguntas (`destino_playa`, `ritmo_nocturna`, `cultura_museos`), mientras que la base de datos de ciudades utiliza otra nomenclatura para sus atributos.

En lugar de modificar alguno de los dos lados, se creó una **capa de traducción** en `CatalogoCiudades`:

```
EQUIVALENCIAS.put("destino_playa", "playa");
EQUIVALENCIAS.put("ritmo_nocturna", "vida_nocturna");
EQUIVALENCIAS.put("cultura_museos", "museos");
```

Esta decisión mantiene ambos sistemas independientes: el cuestionario puede reescribirse sin afectar la base de datos, y viceversa. Si los nombres de los atributos cambian, solo hay que editar esa tabla.

Cuatro claves del cuestionario no tienen equivalencia (`ritmo_descanso`, `ritmo_improvisar`, `ritmo_actividades`, `gastro_restricciones`) porque describen al viajero y no al destino. Actualmente se ignoran en el cálculo.

6.3 Presentación de resultados

Los destinos se muestran como tarjetas con formato de pase de abordar, cada una con la ciudad, el país, el porcentaje de coincidencia total y el desglose entre ajuste presupuestal y coincidencia de gustos.

Cuando no hay destinos disponibles, la pantalla muestra un estado vacío con una acción sugerida (ajustar el presupuesto) en lugar de una página en blanco.

7. Interfaz

El diseño conserva la identidad visual del prototipo React: fondo arena, azul profundo como color de texto, turquesa para las acciones y coral para los errores.

Tipografía. Se usan tres familias con roles diferenciados: Fraunces para títulos, Instrument Sans para el cuerpo del texto e IBM Plex Mono para etiquetas, fechas y cifras. Esta última hace que los datos numéricos se lean como datos y no como prosa.

Sistema de tokens. Los colores y tipografías están definidos como variables CSS en un solo bloque, de modo que un cambio de paleta no requiere buscar y reemplazar en todo el archivo.

Indicador de progreso. Las pantallas del flujo muestran en qué paso se encuentra la persona y cuáles ya completó.

Diseño adaptable. Las vistas se reorganizan en pantallas pequeñas, y se respeta la preferencia del sistema de reducir animaciones.

8. Cómo ejecutar el proyecto

Requisito: JDK 21 o superior. No es necesario instalar Maven ni Tomcat.

Desde la carpeta del proyecto:

```
mvnw.cmd spring-boot:run
```

En Linux o Mac:

```
./mvnw spring-boot:run
```

Cuando aparezca en la consola `Started WisetripApplication`, abrir en el navegador:

```
http://localhost:8090
```

Para generar el archivo distribuible:

```
mvnw.cmd clean package
```

Se produce `target/wisetrip-0.0.1-SNAPSHOT.war`, que se ejecuta con:

```
java -jar wisetrip-0.0.1-SNAPSHOT.war
```

9. Pruebas realizadas

Caso	Resultado esperado
Registro con campos vacíos	Mensajes de error en cada campo
Fecha de nacimiento de menor de edad	"Debes ser mayor de 18 años"
Documento con caracteres no numéricos	Error de formato
Correo ya registrado	"Ya existe una cuenta con este correo"
Contraseñas que no coinciden	Error de confirmación
Login con credenciales incorrectas	Mensaje genérico
Ciudad que no pertenece al país	Error de validación cruzada
Cuestionario incompleto	Conteo de faltantes y marcado visual
Fecha de inicio anterior a hoy	Error de fecha
Regreso anterior al inicio	Error de coherencia
Presupuesto con letras o en cero	Error de formato o de valor
Acceso directo a <code>/origen</code> sin sesión	Redirección a <code>/login</code>
Navegación hacia atrás	Los datos se conservan
Caracteres especiales (ñ, tildes)	Se almacenan y muestran correctamente

10. Decisiones técnicas destacadas

Empaquetado WAR. Obligatorio para el funcionamiento de JSP en Spring Boot.

Codificación UTF-8 forzada. Configurada explícitamente en `application.properties` para garantizar el manejo correcto de la ñ y las tildes en todo el recorrido: formulario, servidor, almacenamiento y vista.

Validación exclusivamente en el servidor. Las validaciones del navegador pueden desactivarse; las del servidor no.

Datos estructurados en lugar de código repetido. El cuestionario, la lista de países y ciudades, y la tabla de monedas están definidos como estructuras de datos, no como bloques de HTML o cadenas de condicionales. Modificarlos no requiere tocar la lógica ni las vistas.

Separación estricta de capas. Los controladores no contienen reglas de negocio ni generan HTML; las vistas no calculan nada. Esta separación es la que permitirá conectar la base de datos modificando únicamente los servicios.

11. Trabajo pendiente

Persistencia en base de datos. Los datos residen en memoria. La migración requiere agregar las dependencias de JPA y el controlador de base de datos, anotar los modelos como entidades y sustituir el `ArrayList` de los servicios por repositorios. La lógica de validación no requiere cambios.

Cifrado de contraseñas. Actualmente se almacenan en texto plano, lo cual es aceptable para almacenamiento en memoria pero inadmisible en base de datos. Debe incorporarse BCrypt al momento de la migración.

Confirmación de los nombres de atributos. La tabla de equivalencias en `CatalogoCiudades` debe verificarse contra los nombres reales de la tabla `atributo`.

Conexión del catálogo de ciudades. El catálogo está definido en código. Al conectar la base, se sustituye por la consulta correspondiente en un único método.

Tratamiento de restricciones. Tres preguntas del cuestionario (restricciones alimentarias, viaje con niños, viaje con mascotas) expresan necesidades y no preferencias. Convendría tratarlas como filtros que descarten destinos incompatibles, en lugar de como un factor más del puntaje.

Filtrado por presupuesto. El selector ordena los destinos pero no descarta los que exceden el presupuesto. El equipo debe definir si deben ocultarse.

Tasas de cambio actualizadas. Las tasas son fijas y aproximadas. Sustituirlas por una consulta a una API externa requeriría modificar un solo método.

